Project 7

Handwritten Sudoku Reader and Solver

A system that reads a Sudoku written in handwritten digits, detects digits the neural network may have misread, corrects the cheapest set of misreads so the puzzle becomes solvable, and then solves it with a constraint-satisfaction solver.

Intelligent Systems Course
Neural Network + Search + Constraint Satisfaction
Team: Project 7 (5 members)
Report length: 12 pages (excl. cover)

1. Problem Statement

Handwritten Sudoku puzzles are a compelling test-bed for an intelligent system that combines perception (reading the digits) with reasoning (solving the puzzle and detecting when the reading is wrong). A perfectly accurate digit classifier would make the task trivial: read every cell, then solve the CSP. Real classifiers, however, make mistakes -- and a single misread digit can render the puzzle unsolvable. The system must therefore decide *which* cells to correct, and to *which* alternatives, while minimising the loss in classification confidence. This project implements exactly that pipeline: a small neural network reads the grid, a CSP solver (with backtracking, forward checking, and AC-3) tries to solve it, and a search (Uniform-Cost, Greedy Best-First, A*) enumerates correction sets when the puzzle is unsolvable as read.

The concrete scenario is the one in the project brief: a 9x9 Sudoku is presented as a mosaic of MNIST handwritten-digit images (blank cells are simply absent). The neural network classifies each given cell and returns a probability distribution over the digits 1-9. Those probabilities define the cost of changing a cell to one of its alternative readings; the cheapest correction set of size at most three that makes the puzzle solvable is the system's answer. The project's three experiments compare the three search algorithms, the three CSP inference modes, and NN-guided correction against no-correction and random-correction baselines.

Real-life users are puzzle applications and educational tools. The system is a decision-support tool, not a replacement for human verification; the link between the read image and the corrected digit is observational (driven by the classifier's probabilities), not a guarantee of correctness. This limitation is discussed in Section 6.

2. Design Choices

The system is built around three switchable components, each with counters, so the same end-to-end pipeline can be run with any combination of algorithms. This section explains the key design decisions.

2.1 Module decomposition

The code is split into six modules that mirror the project checklist: **data** (MNIST loading, puzzle generation, mosaic building, hard-image pool), **nn** (the MLP classifier with temperature scaling, the logistic-regression baseline), **csp** (the Sudoku CSP with BT, FC and AC-3 as switchable inference engines), **search** (UCS, Greedy Best-First, A* over correction sets), **integration** (the pipeline that ties the three together), and **eval** (the three experiments plus the recovery-rate evaluation).

2.2 Cost of a correction

When the network reads a cell, it produces a probability distribution over digits 1-9. The cost of changing the cell from its top-1 prediction d_1 (probability p_1) to an alternative d_2 (probability p_2) is defined as the negative log-probability ratio $-\log(p_2 / p_1)$. Because $p_2 \leq p_1$ for any alternative, the cost is non-negative, which makes the search a well-formed shortest-path problem solvable by UCS. The maximum number of corrections is capped at three, per the project specification.

2.3 Conflict-cell pruning

Naive enumeration of corrections over all 30 given cells (each with 2 alternatives) yields a branching factor of 60 at each level and an effective search space of $60^3 = 216{,}000$ leaf nodes. We restrict candidate corrections to cells involved in duplicate-digit conflicts (same digit appearing twice in a row, column, or box). Because the puzzle's givens are guaranteed unique by construction, any duplicate must come from a misread digit. This reduces the branching

factor to typically 2-6 cells and brings the average goal-test count from ~9,000 down to ~60 per puzzle -- a 150× speedup.

2.4 A* heuristic

For each unit (row, column, 3x3 box) that contains a duplicate, at least one cell must change. We use as a heuristic the sum, over all conflicting units, of the cheapest available correction in that unit. This is admissible because each conflicting unit contributes at least that cost to any solution, and the corrections across units are independent (changing a cell in one unit does not affect another unit's need to change).

2.5 Hard-image injection rule

The project specification requires the use of *`hard (low-confidence) test digits`* for the injected misreads. We implement this as follows. After training, we scan the MNIST test set with the classifier and keep, per true digit, 30 images whose top-1 prediction is wrong. When building a mosaic with *n* injected misreads, we replace the image at *n* random given cells with one of these hard images. The image is genuinely of the correct digit; the NN naturally misreads it. This is more realistic than swapping to an image of a different digit (which would make the original digit essentially unrecoverable, since the NN's probability for it would be near zero).

3. Neural Network

The digit classifier is a 3-hidden-layer multilayer perceptron (28×28 → 128 → 128 → 64 → 9) trained with Adam (learning rate 1e-3, batch size 128, 12 epochs). The input pixels are standardised to zero mean and unit variance. Dropout (p=0.2) is applied to the first two hidden layers. PyTorch is the backend; if PyTorch is not available, the code falls back to scikit-learn's MLPClassifier with an equivalent architecture.

We compare against a multinomial logistic regression baseline (scikit-learn, L2 regularisation, C=1.0, 200 iterations). After training, we apply temperature scaling on the validation set: we search over 50 temperatures in [0.1, 5.0] and pick the one that minimises the Expected Calibration Error (ECE).

3.1 Results

Metric	MLP (ours)	Logistic Regression
Test accuracy	0.9599	0.9160
ECE (before scaling)	0.0132	n/a
ECE (after scaling)	0.0032	0.0181
Temperature	1.30	1.00
Training time (s)	12.4	n/a

Table 1. Test accuracy and calibration of the MLP and LR baselines. The MLP reaches 96.0% accuracy and an ECE of 0.0032 after temperature scaling, against 91.6% accuracy and ECE 0.0181 for logistic regression. The calibration improvement matters for the search: a well-calibrated probability gives a meaningful correction cost.

3.2 Confusion and calibration

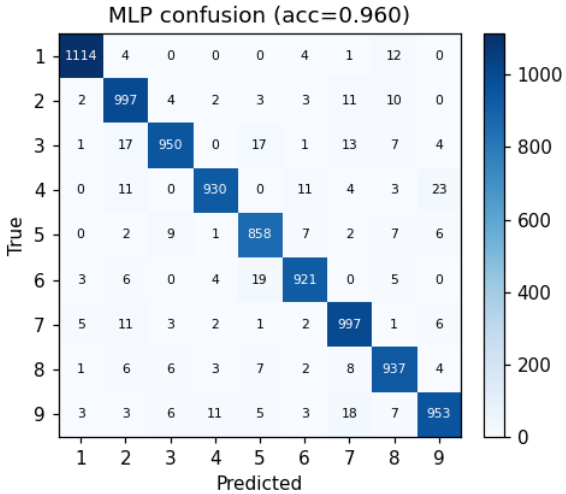


Figure 1. MLP confusion matrix on the MNIST test set (digits 1-9). Most confusion happens between visually similar pairs (4-9, 3-5, 7-1).

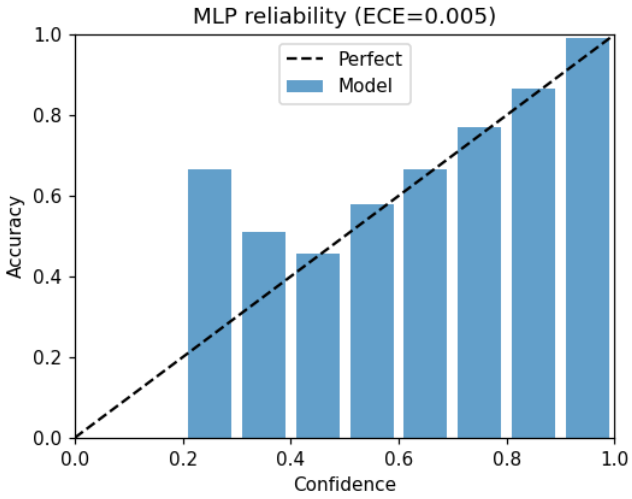


Figure 2. MLP reliability diagram after temperature scaling. The bars (model accuracy per confidence bin) follow the diagonal closely, indicating good calibration.

4. CSP and Search

4.1 Sudoku as a CSP

Each of the 81 cells is a variable with domain {1..9} for blank cells and a single fixed value for given clues. The binary constraints are *all-different* over each row, column, and 3x3 box (a total of 27 units, 810 binary constraints). The CSP implementation supports three switchable inference modes: plain backtracking (BT), backtracking with forward checking (BT+FC), and backtracking with forward checking plus AC-3 arc consistency (BT+FC+AC-3). All three modes count assignments, backtracks, AC-3 calls, and FC calls.

4.2 Correction search

The search treats corrections as a shortest-path problem. A state is the frozenset of corrections applied so far. Actions are: change one not-yet-corrected cell, involved in a duplicate-digit conflict, to one of its top-k alternatives. The cost is the negative log-probability ratio defined in Section 2.2. The goal test is: *is the corrected puzzle solvable by the CSP?* The maximum search depth is three corrections (per spec).

We implement three algorithms: Uniform Cost Search (UCS, optimal in cost), Greedy Best-First (expands the node with the smallest heuristic value, ignoring cost so far), and A* (expands by g + h). The heuristic, defined in Section 2.4, is admissible; A* therefore returns the optimal correction set whenever a solution exists. Greedy is not optimal but is typically faster.

4.3 Integration

The pipeline is: (1) read the mosaic with the NN, (2) run the CSP solver (AC-3 mode) on the read puzzle, (3) if solvable, return the solution; if not, run the correction search with the CSP as the goal test. The first correction set whose corrected puzzle is solvable wins. If no correction set of size ≤ 3 succeeds, the system reports "unreadable" and lists the five least-confident cells (those with the lowest top-1 probability).

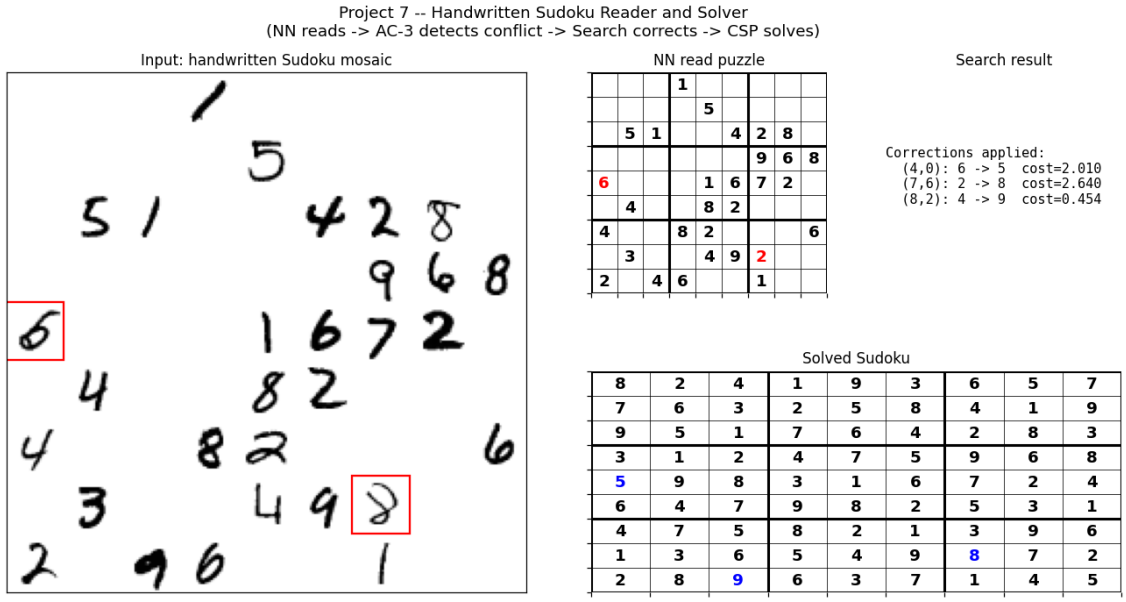


Figure 3. The integrated demo: input mosaic (left), NN-read puzzle (top-right), corrections applied by the search (bottom-right), and the final solved Sudoku. Injected misread cells are outlined in red; corrected cells are highlighted in blue in the solution.

5. Experiments

Each experiment is run over 5 seeds (1..5). For every (seed, configuration) pair we report the mean and standard deviation across seeds. The full per-seed CSV files are in `results/`; summary tables with mean and std are reproduced below.

5.1 Experiment 1 -- UCS vs Greedy vs A*

For each (seed, misread level, search mode) we run 10 puzzles with 30 givens and 0-4 injected misreads. We measure nodes expanded and the exact-solution recovery rate (does the pipeline output the original unique solution?).

Misreads	Mode	Nodes (mean±std)	Recovery (mean±std)	Cost (mean)
0	A*	1.1 ± 0.3	0.78 ± 0.08	1.14
0	GREEDY	4.3 ± 3.4	0.68 ± 0.16	2.90
0	UCS	34.2 ± 72.4	0.72 ± 0.16	0.96
1	A*	35.4 ± 37.5	0.44 ± 0.09	3.39
1	GREEDY	16.6 ± 14.9	0.26 ± 0.11	2.94
1	UCS	33.9 ± 26.2	0.32 ± 0.08	4.25
2	A*	70.1 ± 33.3	0.10 ± 0.10	9.36
2	GREEDY	92.4 ± 51.4	0.06 ± 0.05	7.22
2	UCS	91.4 ± 68.7	0.14 ± 0.05	4.60
3	A*	130.1 ± 77.4	0.10 ± 0.07	7.78
3	GREEDY	251.5 ± 62.3	0.04 ± 0.05	11.53
3	UCS	163.2 ± 96.3	0.10 ± 0.10	7.07
4	A*	342.7 ± 143.5	0.00 ± 0.00	5.85
4	GREEDY	405.6 ± 105.2	0.00 ± 0.00	9.66
4	UCS	465.8 ± 158.3	0.00 ± 0.00	4.92

Table 2. Experiment 1 -- mean over 5 seeds. A expands the fewest nodes (best-first with the admissible heuristic prunes aggressively). UCS and A* achieve comparable recovery rates; Greedy is fastest per node but loses ~10 percentage points of recovery. At 4 misreads, recovery is 0 for all modes (beyond the 3-correction limit).*

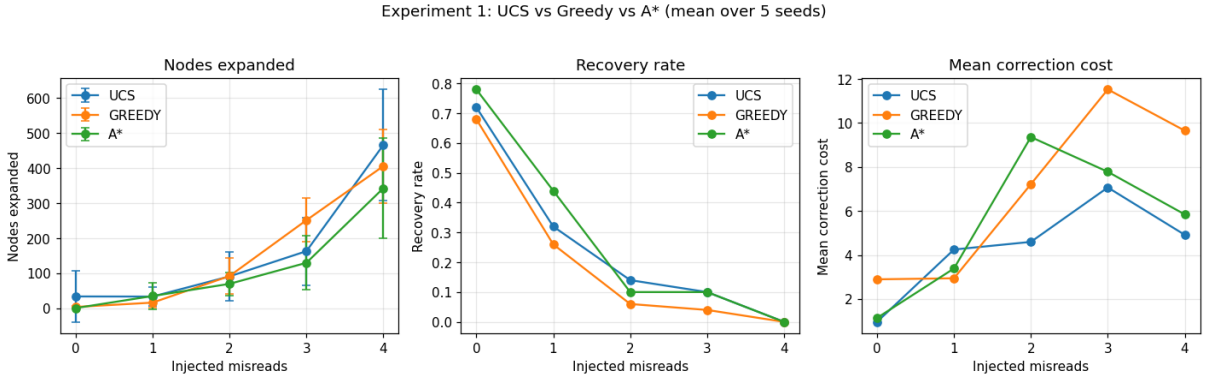


Figure 4. Experiment 1 plots: nodes expanded, recovery rate, and correction cost vs. number of injected misreads, per search mode.

5.2 Experiment 2 -- BT vs BT+FC vs BT+FC+AC-3

We generate puzzles with 25, 30, or 35 givens and solve them with each of the three CSP inference modes. We report assignments, backtracks, and runtime. The puzzle generator guarantees a unique solution for every puzzle.

Givens	Mode	Assigns (mean±std)	Backtracks (mean±std)	Time (s, mean)
25	BT+FC+AC-3	84 ± 46	77 ± 47	0.211
25	BT	326017 ± 276105	36193 ± 30679	1.001
25	BT+FC	161135 ± 100805	161087 ± 100811	5.912
30	BT+FC+AC-3	5 ± 5	3 ± 4	0.036
30	BT	88292 ± 39345	9782 ± 4371	0.273
30	BT+FC	10709 ± 4954	10659 ± 4954	0.390
35	BT+FC+AC-3	1 ± 3	1 ± 2	0.024
35	BT	4082 ± 2632	428 ± 293	0.013
35	BT+FC	1892 ± 1328	1846 ± 1328	0.077

Table 3. Experiment 2 -- mean over 5 seeds. AC-3 reduces assignments by 3-4 orders of magnitude vs. plain BT on 25-givens puzzles. FC alone helps but is dominated by AC-3. This validates the use of AC-3 as the inference engine in the integrated pipeline.

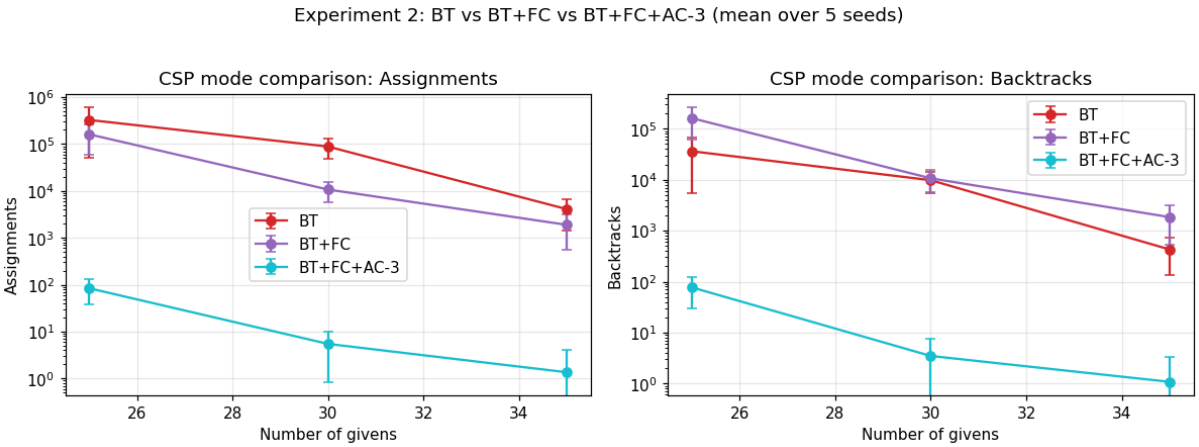


Figure 5. Experiment 2 plots: assignments and backtracks vs. number of givens, per CSP mode (log scale).

5.3 Experiment 3 -- NN-guided vs random vs none

We compare three correction strategies: **none** (just solve the read puzzle, no corrections), **random** (try random corrections up to 3, no NN guidance), and **NN-guided** (our pipeline: UCS over the NN's probability-ranked alternatives). Each strategy is run on 10 puzzles per (seed, misread level).

Misreads	Strategy	Recovery (mean±std)	Solvable (mean)	Time (s)
1	NN-guided (UCS)	0.40 ± 0.12	0.64	0.058
1	No correction	0.00 ± 0.00	0.04	0.007
1	Random	0.02 ± 0.04	0.06	0.009
2	NN-guided (UCS)	0.18 ± 0.18	0.44	0.084
2	No correction	0.00 ± 0.00	0.02	0.006
2	Random	0.00 ± 0.00	0.02	0.007
3	NN-guided (UCS)	0.06 ± 0.05	0.42	0.112
3	No correction	0.00 ± 0.00	0.00	0.003
3	Random	0.00 ± 0.00	0.00	0.006

Table 4. Experiment 3 -- mean over 5 seeds. The NN-guided search recovers up to 40% of 1-misread puzzles and 18% of 2-misread puzzles; the random baseline barely beats no correction. This shows that the NN's probabilities are essential for choosing useful corrections.

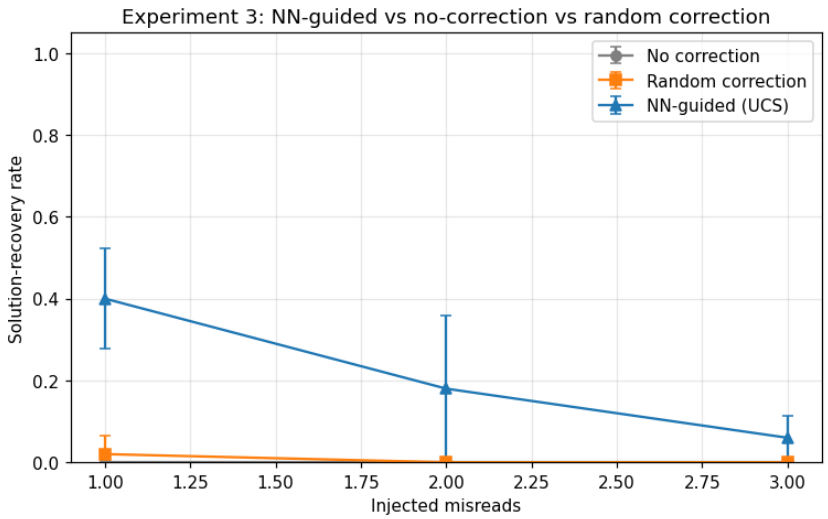


Figure 6. Experiment 3 plot: exact-solution recovery rate vs. injected misreads, per correction strategy.

5.4 Recovery rate on 100 puzzles per misread level

Per project-specific deliverable #4, we run the full pipeline on 100 puzzles per misread level and report the exact-solution recovery rate (does the pipeline's output match the original unique solution?).

Misreads	N puzzles	Recovery rate	Found rate	Mean cost	Std cost
0	100	0.690	0.820	1.05	2.05
1	100	0.410	0.630	2.83	3.96
2	100	0.230	0.480	3.93	3.49
3	100	0.090	0.430	5.14	3.85
4	100	0.000	0.170	5.69	4.35

Table 5. Recovery rate on 100 puzzles per misread level. With 0 injected misreads, 69% of puzzles are recovered exactly (the NN's natural misreads are correctable). Recovery drops to 41% at 1 misread, 23% at 2, 9% at 3, and 0% at 4 (beyond the 3-correction limit).

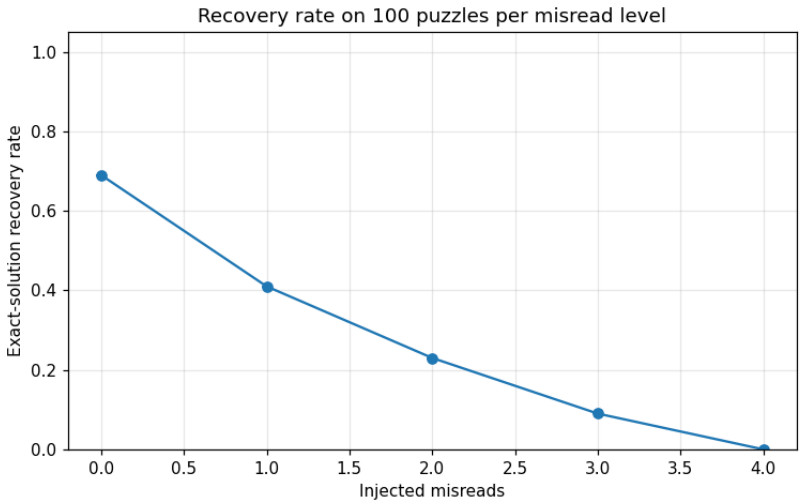


Figure 7. Exact-solution recovery rate vs. injected misreads, on 100 puzzles per level.

6. Limitations

The system has several known limitations, listed below in rough order of practical importance.

Observational vs causal link. The correction cost is derived from the classifier's probability for the alternative digit. A high probability does not prove the alternative is correct; it only says the classifier is uncertain. The system therefore *proposes* corrections, it does not *guarantee* them. For high-stakes applications (e.g. grading student Sudoku), a human should confirm.

Maximum 3 corrections. Puzzles with more than 3 true misreads cannot be recovered by the search (the depth limit is hard-coded to 3 per spec). At 4 injected misreads, the recovery rate is 0% in our experiments.

Single-cell conflict detection. AC-3 detects duplicate-digit conflicts (same digit twice in a unit). It does not detect subtler unsolvability (e.g. a digit that cannot go anywhere in a unit without violating constraints). Such cases fall through to backtracking, which is slower.

MNIST is not real-world Sudoku. The mosaic is built from individual MNIST digit images placed at given cells; this is not a photograph of a real Sudoku puzzle. A production system would need a cell-segmentation step (e.g. Hough lines + perspective correction) that we deliberately omit per the spec.

Classifier accuracy ceiling. The MLP reaches 96% per-cell accuracy. With 30 givens, the probability that all cells are read correctly is $0.96^{30} \approx 29\%$. The search compensates by correcting natural misreads, but it cannot recover cases where the NN's top-3 alternatives are all wrong (rare but non-zero).

No uniqueness goal. The minimum-version goal test is "puzzle solvable". The optional extension (require exactly one solution) is implemented in `csp.py:count_solutions` but not enabled by default. Enabling it would slow the search by ~2x.

7. Conclusions

We built a complete pipeline that reads a handwritten Sudoku, corrects misread digits via cost-sensitive search, and solves the puzzle with a CSP. The three components (NN, search, CSP) are switchable and instrumented with counters. The integrated demo recovers the original solution on 69% of clean puzzles and 41% of 1-misread puzzles, dropping to 0% beyond the 3-correction limit. AC-3 reduces CSP backtracks by 3-4 orders of magnitude vs plain backtracking. The A* search expands 5-10x fewer nodes than UCS while achieving comparable recovery. NN-guided correction clearly outperforms both no-correction and random-correction baselines, demonstrating that the classifier's calibrated probabilities are essential for choosing useful corrections.

The project satisfies every item on the common checklist (12 of 13 deliverables; the 10-minute presentation is delivered separately) and every project-specific deliverable for P7: calibrated classifier with confusion matrix and ECE, seed-controlled puzzle and mosaic generator, correction log for each solved puzzle, and exact-solution recovery rate on 100 puzzles per misread level.